WEEK 2 · LESSON 12 OF 24 · HANDOUT

Week 2 Project: Monitor, Detect and Retrain

Study time: **about 120 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=4BgmBtsS0iA — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon DE 2022: Detecting drift — how to evaluate and explore data drift in ML systems — https://www.youtube.com/watch?v=0wdCMvU1vCQ

🎓 **Go deeper:** PyData Eindhoven 2019: Managing the Machine Learning Lifecycle with MLflow — https://www.youtube.com/watch?v=hm3HIZ5mns0

Your goal in this lesson: Run a model through six simulated months of production, detect drift and quality loss, retrain on recent data, and decide on promotion with evidence.

The brief

The loan model went live in January. Each month you receive the new applications, and — later — whether they repaid. The economy slowly changes (data drift), and from March the rules of repayment also change (concept drift). Your job: monitor, decide when to act, retrain, and prove the new model is better before promoting it.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

Part 1 — A monthly health check

```
from scipy.stats import ks_2samp

def psi(reference, current, bins=10):
    edges = np.unique(np.quantile(reference, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    ref = np.clip(np.histogram(reference, edges)[0] / len(reference), 1e-4, None)
    cur = np.clip(np.histogram(current, edges)[0] / len(current), 1e-4, None)
    return float(np.sum((cur - ref) * np.log(cur / ref)))

BASELINE_ACC = accuracy_score(test[TARGET], proba >= 0.5)

def health(model, month_data, reference):
    p = model.predict_proba(month_data[FEATURES])[:, 1]
    psis = {f: psi(reference[f], month_data[f]) for f in FEATURES}
    worst = max(psis, key=psis.get)
    return {"approve_rate": round((p >= 0.5).mean(), 3), "max_psi": round(psis[worst], 3),
"worst_feature": worst,
            "accuracy": round(accuracy_score(month_data[TARGET], p >= 0.5), 3)}
```

Part 2 — Six months in production

```
MONTHS = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
DRIFT = [0.0, 0.05, 0.1, 0.15, 0.2, 0.25]
CONCEPT = [0.0, 0.0, 0.4, 0.8, 1.2, 1.5]

history, months = [], {}
for i, name in enumerate(MONTHS):
    data = make_applicants(n=1200, seed=500 + i, drift=DRIFT[i], concept=CONCEPT[i])
    months[name] = data
    h = health(model, data, train)
    h["accuracy_drop"] = round(BASELINE_ACC - h["accuracy"], 3)
    reasons = []
    if h["max_psi"] > 0.25:
        reasons.append(f"input drift in {h['worst_feature']}")
    if h["accuracy_drop"] > 0.05:
        reasons.append("accuracy down more than 5 points")
    h["action"] = "RETRAIN: " + " and ".join(reasons) if reasons else "ok"
    history.append({"month": name, **h})

report = pd.DataFrame(history)
print(f"baseline accuracy {BASELINE_ACC:.3f}")
print(report.to_string(index=False))
```

```
import matplotlib.pyplot as plt
fig, ax1 = plt.subplots(figsize=(8, 3.5))
ax1.plot(report["month"], report["accuracy"], marker="o", label="accuracy")
ax1.axhline(BASELINE_ACC - 0.05, color="red", linestyle="--", label="retrain line")
ax2 = ax1.twinx(); ax2.bar(report["month"], report["max_psi"], alpha=0.25, color="gray",
label="max PSI")
ax1.set_ylabel("accuracy"); ax2.set_ylabel("max PSI"); ax1.legend(loc="lower left"); plt.show()
```

Read the table: when did input drift appear, and when did accuracy cross the retrain line? Remember the outcomes for a month are only known later — in real life you would see June's accuracy months after June.

Part 3 — Retrain on recent data

```
from sklearn.base import clone

recent = pd.concat([months["May"], months["Jun"]])
recent_train, recent_check = train_test_split(recent, test_size=0.3, random_state=0,
stratify=recent[TARGET])
candidate = clone(model).fit(recent_train[FEATURES], recent_train[TARGET])

july = make_applicants(n=1500, seed=600, drift=0.3, concept=1.6)      # next month, never seen
def score(m, data):
    p = m.predict_proba(data[FEATURES])[:, 1]
    return {"accuracy": round(accuracy_score(data[TARGET], p >= 0.5), 3), "auc":
round(roc_auc_score(data[TARGET], p), 3)}

print("current model on July:  ", score(model, july))
print("retrained model on July:", score(candidate, july))
coef = pd.DataFrame({"feature": FEATURES,
                     "old": model[-1].coef_[0].round(2), "new":
candidate[-1].coef_[0].round(2)})
print(coef.to_string(index=False))
```

The retrained model learned the new rules — look at how the weights for `salaried` and `late_payments` changed. That is concept drift, visible in the model itself.

Part 4 — Decide

```
old_s, new_s = score(model, july), score(candidate, july)
decision = ("PROMOTE 1.1.0 (via shadow mode, then canary)" if new_s["auc"] > old_s["auc"] +
0.01
            else "KEEP 1.0.0 and investigate")
print(decision)
```

Your report (two pages)

1. The monthly table and chart, with one sentence per month.
2. Which problems were data drift and which concept drift, and how you could tell.
3. The retraining: data used, results on July, how the weights changed.
4. The release plan: shadow mode → canary → full, and the rollback trigger.
5. What you would monitor next, and how often.

Marking guide (self-check)

Area	Excellent work
Monitoring	Monthly drift and accuracy with clear thresholds
Diagnosis	Correctly separates data drift from concept drift
Retraining	Recent data, fair comparison on unseen data
Decision	Evidence-based promotion with a safe release plan
Communication	Short, clear explanations for a manager

Remember this: Monitoring, diagnosis, retraining and careful promotion form one loop — the heart of running models in production.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–4 and the report. Then try a different retraining window (only June, or March–June) and explain which works best on July and why.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why can you not see June's accuracy at the end of June in real life?
   - A. The model is offline
   - B. Accuracy is secret
   - C. Repayment outcomes arrive months later
   - D. June is too short
2. Which data did the project retrain on?
   - A. No data
   - B. Random internet data
   - C. Recent months (May and June)
   - D. Only January
3. Why test both models on July data?
   - A. July is longer
   - B. It is easier
   - C. It is new, unseen data — a fair comparison
   - D. To make the old model win
4. What did the changed weights for salaried and late_payments show?
   - A. Data deletion
   - B. Concept drift: the rules of repayment changed
   - C. Nothing
   - D. A bug
5. How should the retrained model be released?
   - A. Shadow mode, then canary, then full rollout with a rollback trigger
   - B. Never
   - C. Email it to customers
   - D. Switch everyone instantly
6. What is the main loop of running models in production?
   - A. Monitor, diagnose, retrain and promote carefully
   - B. Only write reports
   - C. Train once and forget
   - D. Only add servers

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.